

Recap: Exact Line Search

Step 1. Initialize $\mathbf{w}(0)$

Step 2. For iteration $k = 0, 1, 2, \dots$

Repeat until some stopping criterion is met;

(a) Pick a descent direction to go $\mathbf{p}(k)$, $\mathbf{p}(k) = -\nabla f(\mathbf{w}(k))$

(b) Pick the learning rate α_k by solving $\alpha_k = \arg \min_{\alpha > 0} \{\phi(\alpha) = f(\mathbf{w}(k) + \alpha \mathbf{p}(k))\}$

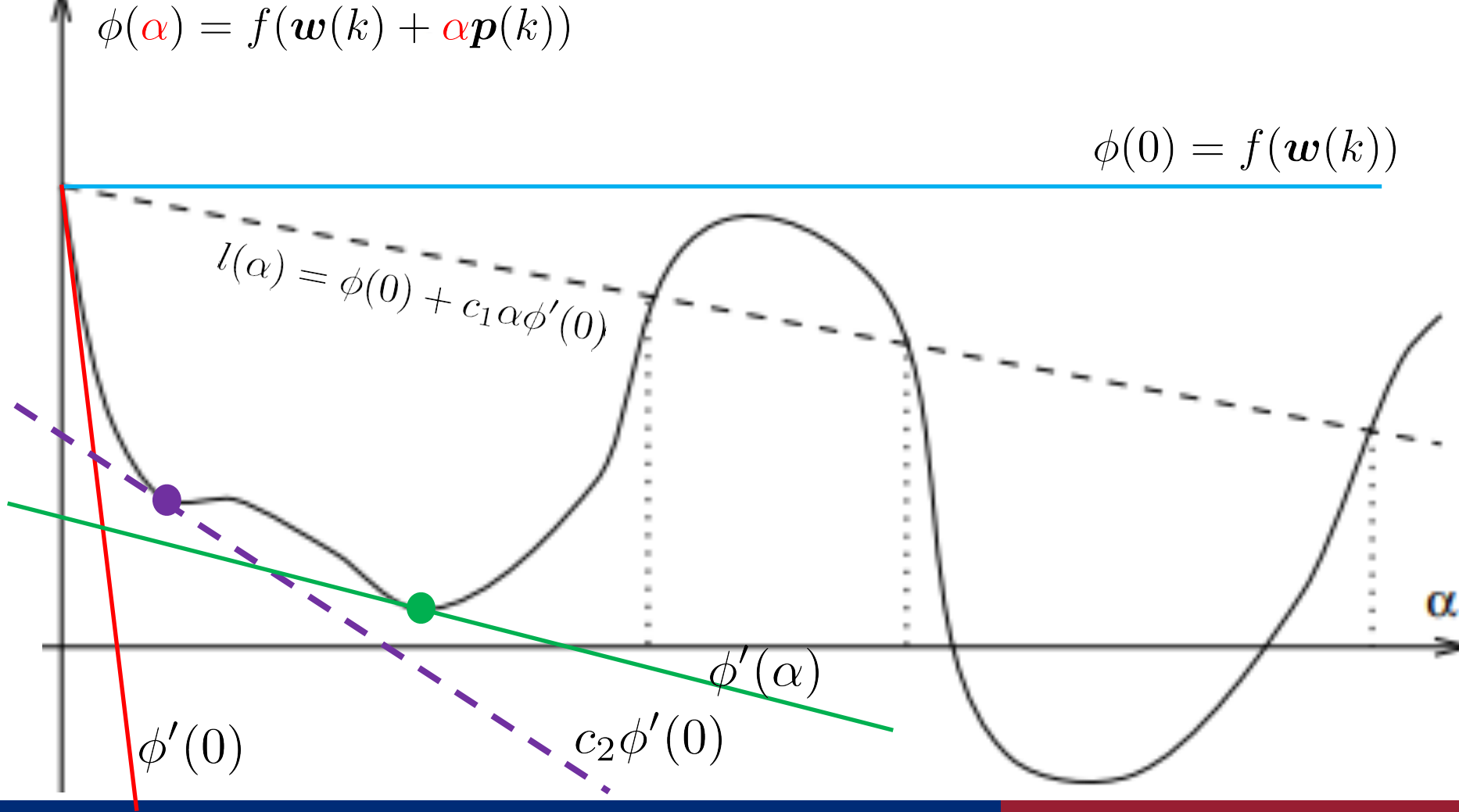
(c) Update $\mathbf{w}(k+1) = \mathbf{w}(k) + \alpha_k \mathbf{p}(k)$

Termination: Stop when $\|\nabla f(\mathbf{w}(k))\| \leq \epsilon$

Ensures *Global Convergence*:
Starting from any initial point, gradient descent with exact line search will lead you toward a stationary point (provided that the gradient is Lipschitz continuous, f is bounded below, etc.)

Recap: Armijo+Curvature = Wolfe Conditions

α cannot be too small or too large. This is because those “bad” cannot achieve “sufficient” reduction in the objective function value within that iteration
Armijo prevents α from being too large and curvature condition prevents α from being too small



Armijo: The “Height” Test

$$\phi(\alpha) \leq \phi(0) + c_1 \alpha \phi'(0)$$

$$c_1 [-\alpha \phi'(0)] \leq \phi(0) - \phi(\alpha)$$

1st order
predicted
decrease

Actual
decrease

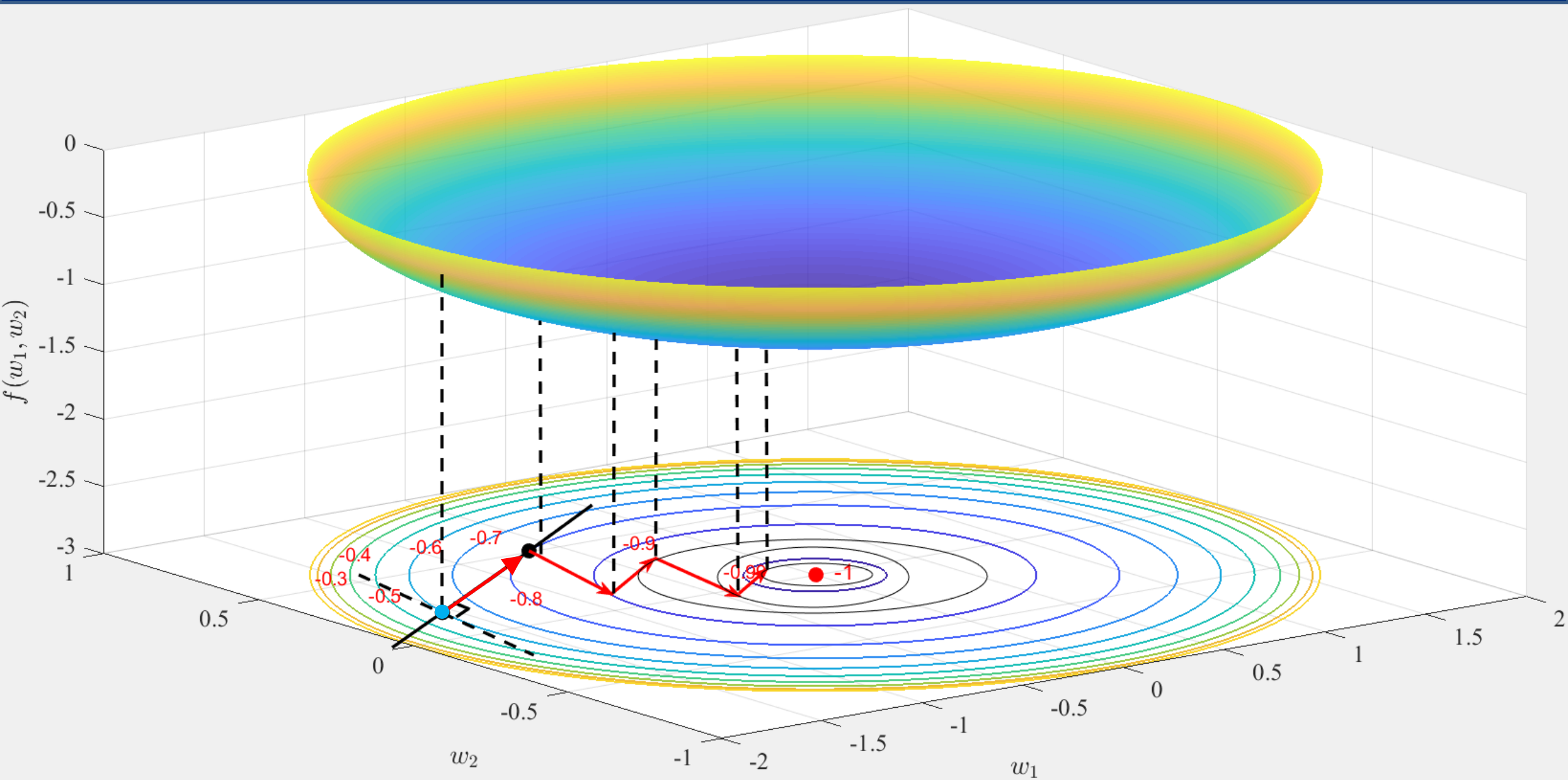
Curvature: The “Slope” Test

$$\phi'(\alpha) \geq c_2 \phi'(0)$$

$$0 < c_1 < c_2 < 1$$

$$e.g., c_1 = 10^{-4}, c_2 = 0.9$$

Recap



Recap

$$\mathbf{w}(k+1) = \mathbf{w}(k) + \mathbf{v}(k+1) \quad \text{where} \quad \mathbf{v}(k+1) = -\alpha \nabla f(\mathbf{w}(k)) + \tau \mathbf{v}(k), \quad \mathbf{v}(0) = \mathbf{0}$$

With momentum, previous gradients are combined with exponentially decaying weights

$$\mathbf{v}(k+1) = -\alpha \nabla f(\mathbf{w}(k)) + \tau \mathbf{v}(k) \\ = -\alpha \nabla f(\mathbf{w}(k)) - \tau \alpha \nabla f(\mathbf{w}(k-1)) - \tau^2 \alpha \nabla f(\mathbf{w}(k-2)) + \tau^3 \mathbf{v}(k-2)$$

current gradient

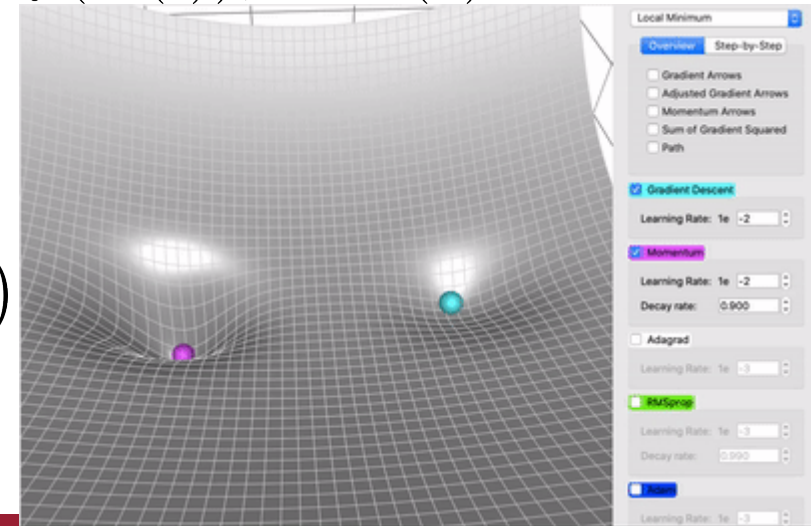
previous
gradients

$$= -\alpha \sum_{i=0}^k \tau^{k-i} \nabla f(\mathbf{w}(i)) + \tau^{k+1} \mathbf{v}(0) = -\alpha \sum_{i=0}^k \tau^{k-i} \nabla f(\mathbf{w}(i)), \quad \because \mathbf{v}(0) = \mathbf{0}$$

Nesterov's Accelerated Gradient Descent (NAG)

$$\mathbf{w}(k+1) = \mathbf{w}(k) + \tau[\mathbf{v}(k+1) - \mathbf{v}(k)] + \mathbf{v}(k+1)$$

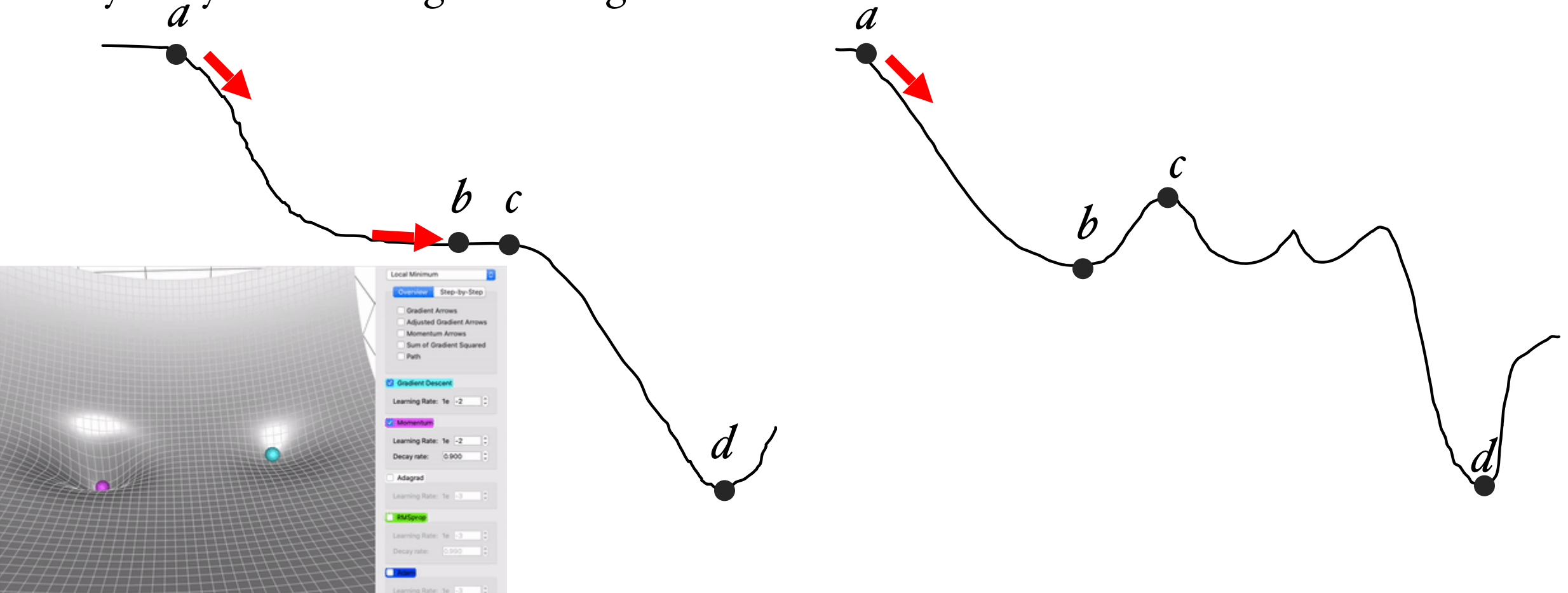
$$\text{where } \mathbf{v}(k+1) = -\alpha \nabla f(\mathbf{w}(k)) + \tau \mathbf{v}(k)$$



Momentum vs. Random Restarts

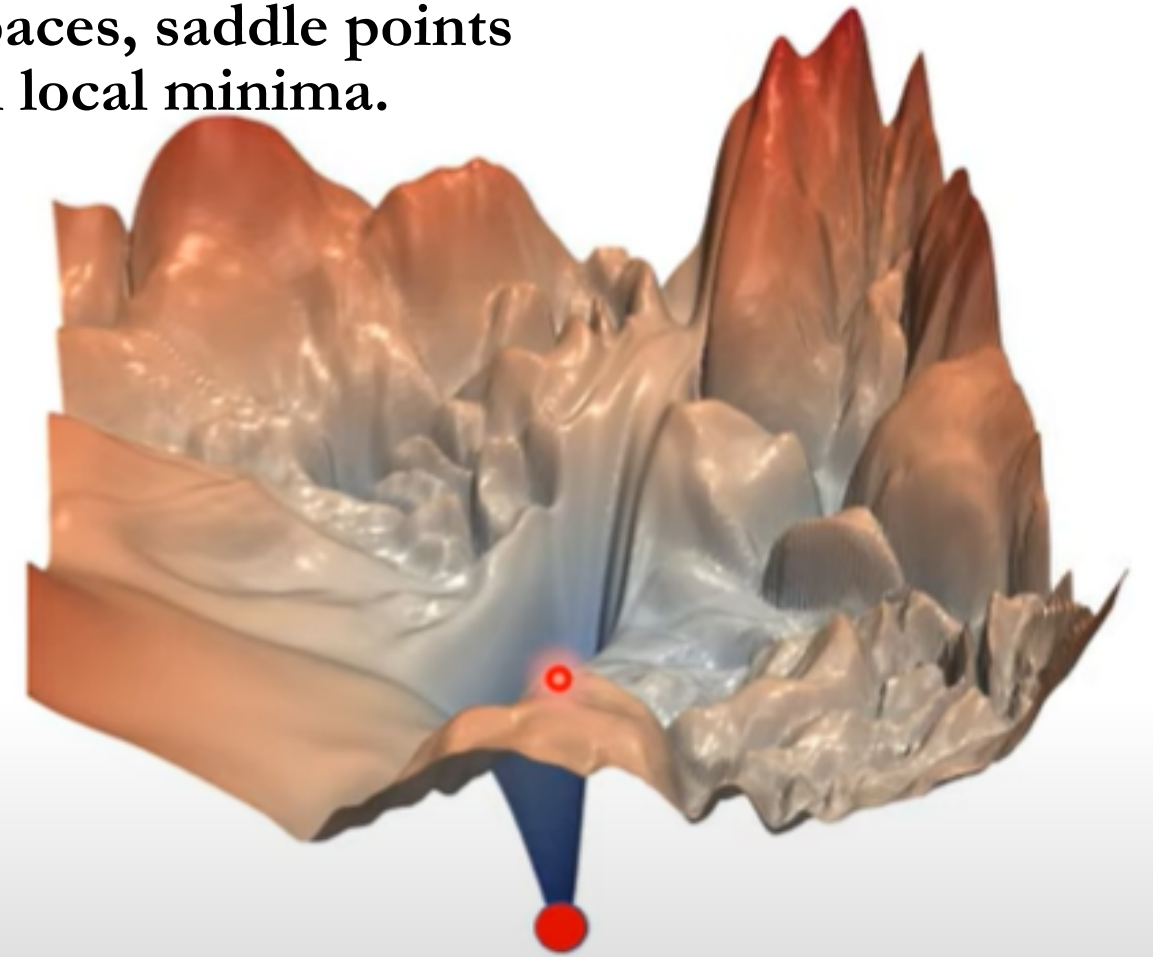
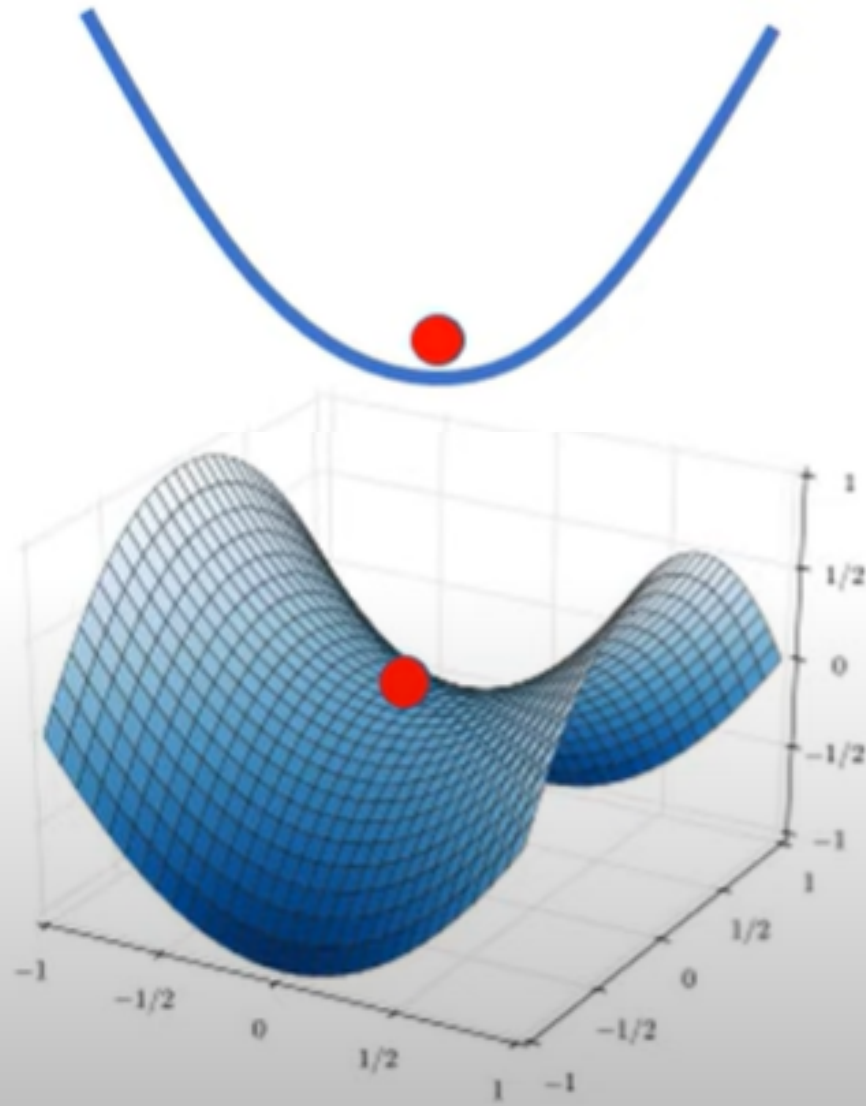
Random restarts launch independent runs from different starting points. They do not help any individual run escape; instead, they improve the chance that at least one run starts in a better basin.

Momentum uses the current gradient and the trajectory's history. Its accumulated velocity may carry a run through a flat region or across a shallow local minimum.



Saddle Points Vs. Local Minima (in a High-Dimensional Space)

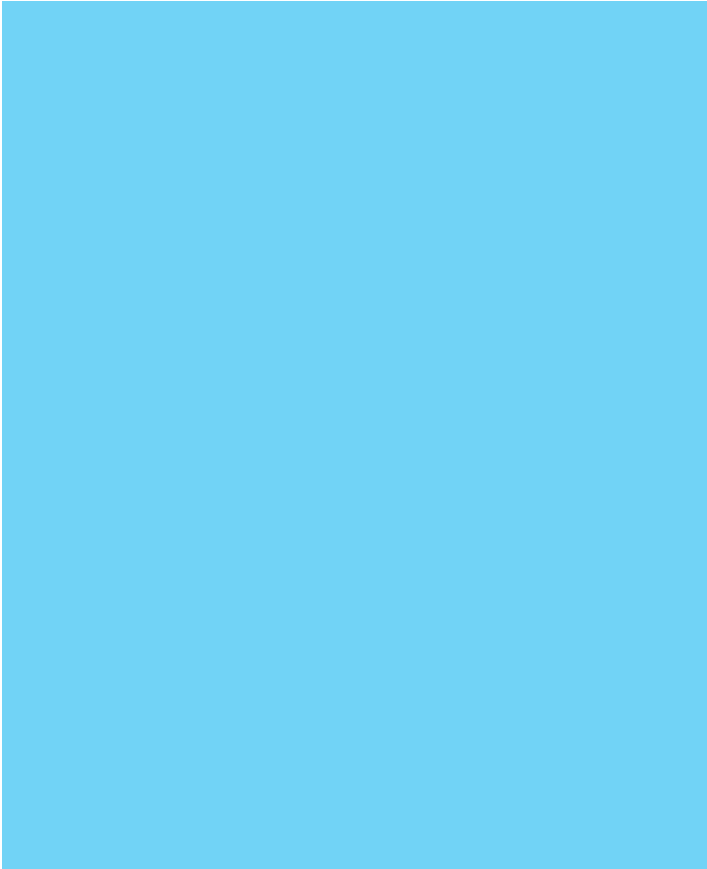
In high-dimensional spaces, saddle points are more common than local minima.



Saddle point in
higher dimension?

Unconstrained Optimization: e.g., Logistic Model

Training Dataset (m rows
and $d+1$ columns)

x_1	x_2	x_3	$\dots$	x_d	y
					1
					-1
					-1
					1
					1
					1
					1
					-1
					-1
					-1
					1
					-1

$m \times d$



Model: $g(\mathbf{w}|\mathbf{x})$

where $\mathbf{w}$ is a set of parameters
that need to be determined as a function of $\mathbf{x}$

$$\mathbf{x} = (x_1, x_2, \dots, x_d)$$

$$Pr\{y|\mathbf{x}\} = \begin{cases} \frac{e^{\mathbf{w}^T \mathbf{x}}}{1 + e^{\mathbf{w}^T \mathbf{x}}}, & \text{if } y = 1, \\ 1 - \frac{e^{\mathbf{w}^T \mathbf{x}}}{1 + e^{\mathbf{w}^T \mathbf{x}}}, & \text{if } y = -1. \end{cases}$$

$$\mathbf{w}^T \mathbf{x} = w_1 x_1 + w_2 x_2 + \dots + w_d x_d$$

$$\min_{\mathbf{w}} f(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m \ln \left(1 + e^{-y^{(i)} \mathbf{w}^T \mathbf{x}^{(i)}} \right)$$

The objective
function f is an
average of m
functions



What if m is
really large?

$$\min_{\mathbf{w}} f(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m h_i(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)})$$

Unconstrained Optimization: e.g., Logistic Model

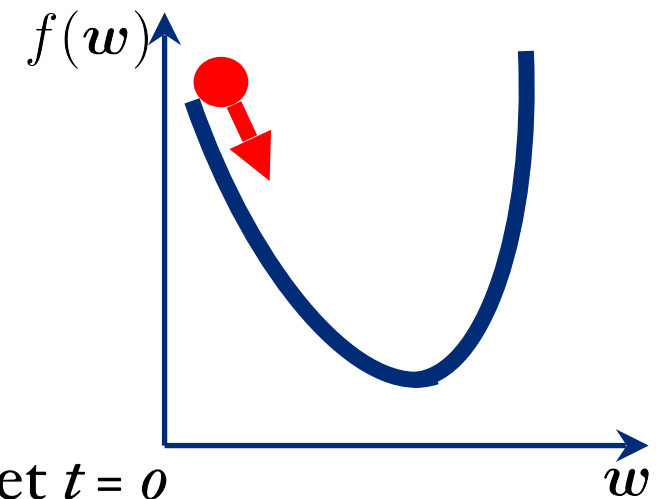
Step 1. Initialize $\mathbf{w}(0)$ (you can randomly pick a vector)

Step 2. for iteration $k = 0, 1, 2, \dots$

Compute the **full** gradient

$$\nabla f(\mathbf{w}(k)) = -\frac{1}{m} \sum_{i=1}^m \frac{y^{(i)} \mathbf{x}^{(i)}}{1 + e^{y^{(i)} \cdot \mathbf{w}^T(k) \mathbf{x}^{(i)}}}$$

Update: $\mathbf{w}(k+1) = \mathbf{w}(k) - \alpha_k \nabla f(\mathbf{w}(k))$, where α_k is:



Backtracking with
Armijo Condition

- a. Initialize $\alpha^{(0)} = 1$ (or some positive number) and set $t = 0$
- b. Until $f(\mathbf{w}(k) + \alpha^{(t)} \mathbf{p}(k)) < f(\mathbf{w}(k)) + \alpha^{(t)} c \cdot \mathbf{p}^T(k) \nabla f(\mathbf{w}(k))$
do the following:
 - (i) Set $\alpha^{(t+1)} = \rho \alpha^{(t)}$, where $\rho \in (0, 1)$, e.g., $\rho = 1/2$
 - (ii) Increase t by 1
- c. Set $\alpha_k = \alpha^{(t)}$

Termination: Stop until $\|\nabla f(\mathbf{w}(k))\|$ is small or k is beyond a cutoff.

Stochastic Gradient Descent (SGD)

Computing the *(full/exact)* gradient $\nabla f(\mathbf{w}(k)) = -\frac{1}{m} \sum_{i=1}^m \frac{y^{(i)} \mathbf{x}^{(i)}}{1 + e^{y^{(i)} \cdot \mathbf{w}^T(k) \mathbf{x}^{(i)}}}$ is the most costly work

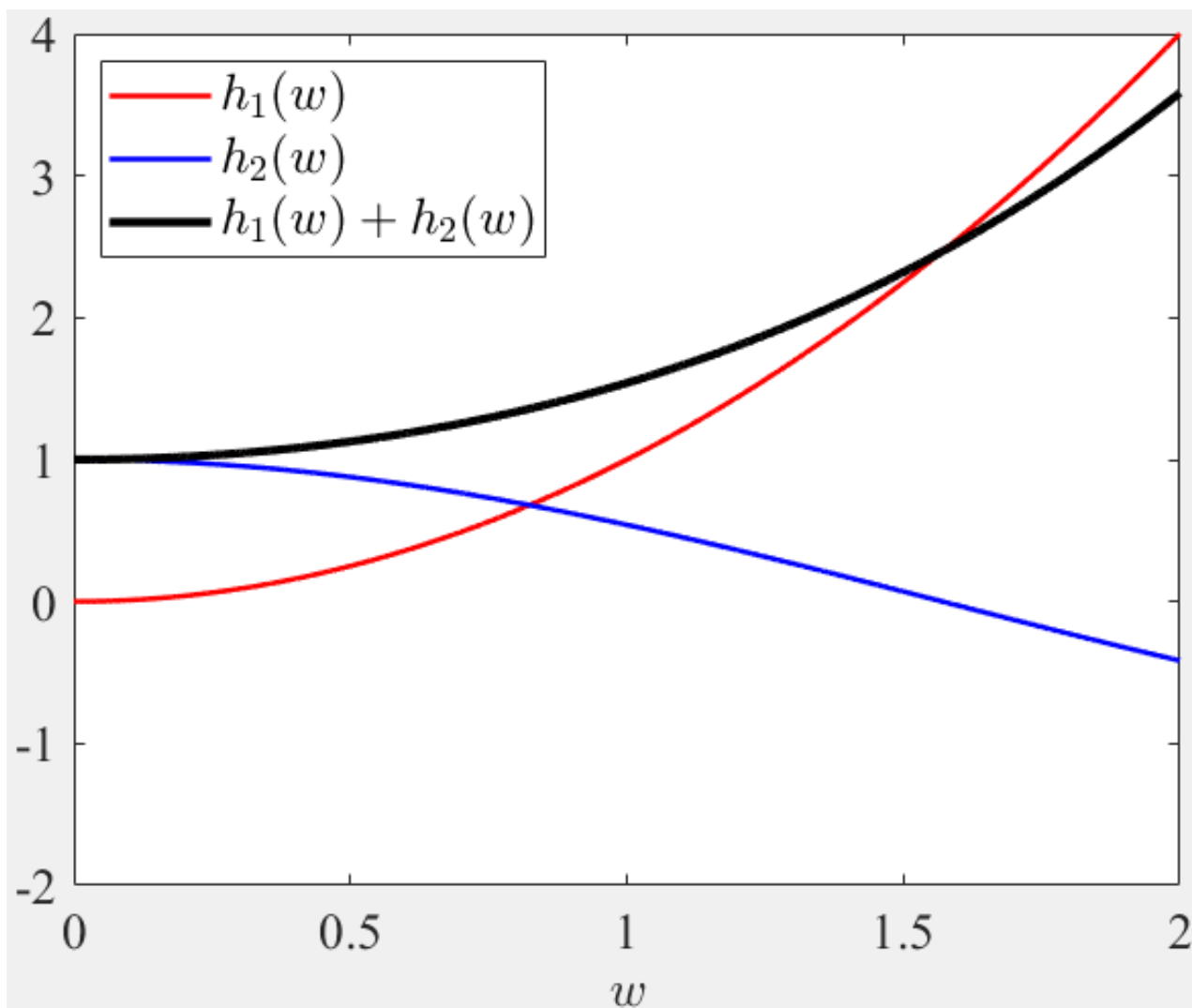
$$\begin{aligned} f(\mathbf{w}) &= \frac{1}{m} \sum_{i=1}^m h_i(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) \quad \Rightarrow \quad \nabla f(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m \nabla h_i(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) \\ &= \frac{1}{m} \left[\nabla h_1(\mathbf{w}) + \nabla h_2(\mathbf{w}) + \cdots + \nabla h_m(\mathbf{w}) \right] \end{aligned}$$

Need to compute m gradients!
This is very costly when m is large.

Stochastic Gradient Descent (SGD)

$m = 2$ and $w \in \mathbb{R}$

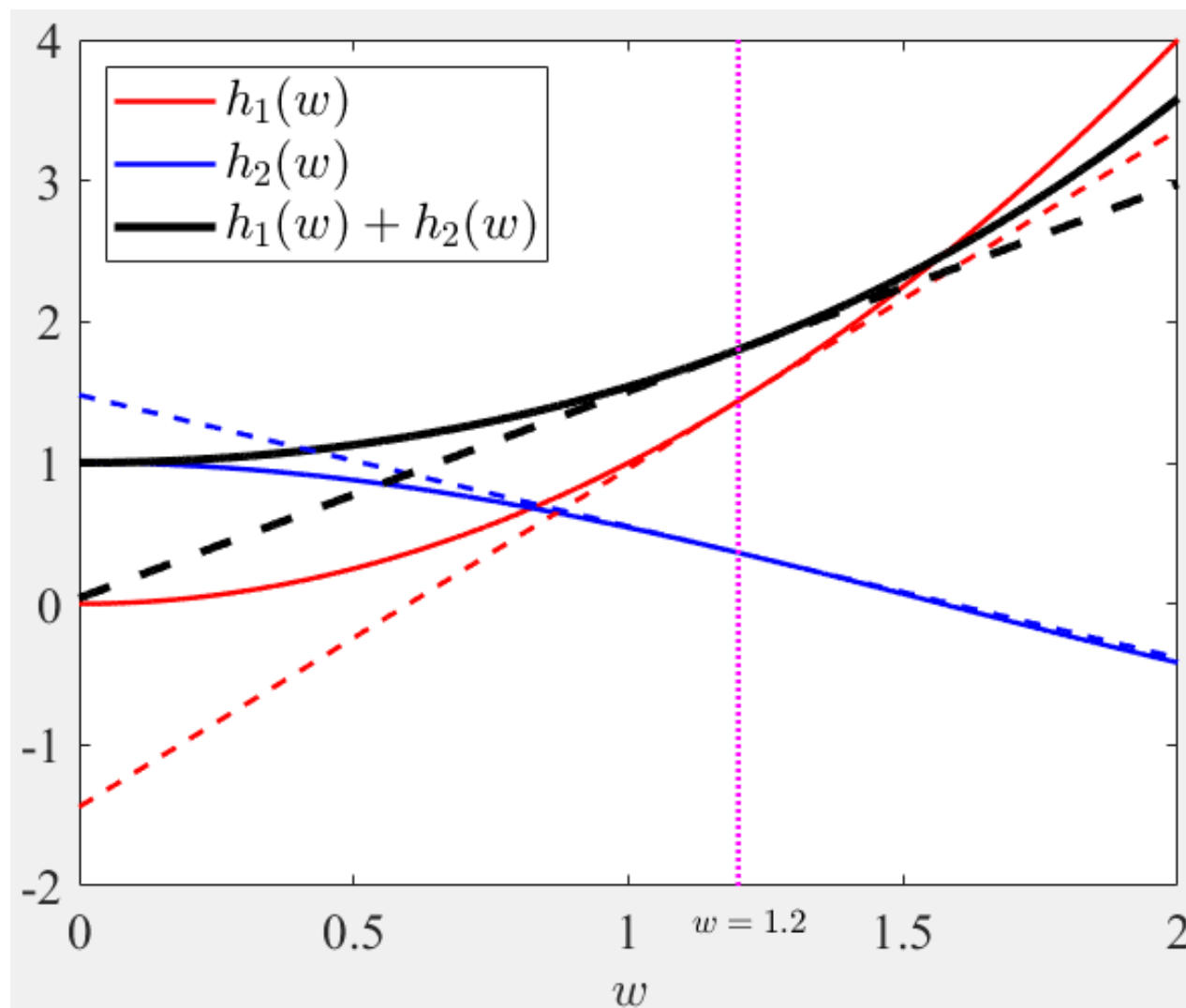
$$f(w) = h_1(w) + h_2(w)$$



Stochastic Gradient Descent (SGD)

$m = 2$ and $w \in \mathbb{R}$

$$\nabla f(w) = \nabla h_1(w) + \nabla h_2(w)$$



Stochastic Gradient Descent (SGD)

Computing the *(full/exact)* gradient $\nabla f(\mathbf{w}(k)) = -\frac{1}{m} \sum_{i=1}^m \frac{y^{(i)} \mathbf{x}^{(i)}}{1 + e^{y^{(i)} \cdot \mathbf{w}^T(k) \mathbf{x}^{(i)}}}$ is the most costly work

$$f(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m h_i(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)})$$

$$\Rightarrow \nabla f(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m \nabla h_i(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) = \frac{1}{m} \left[\nabla h_1(\mathbf{w}) + \nabla h_2(\mathbf{w}) + \cdots + \nabla h_m(\mathbf{w}) \right]$$

Need to compute m gradients! This is very costly when m is large.

What about we *randomly* pick a function $h_i(\mathbf{w})$ among $\{h_i(\mathbf{w})\}_{i=1}^m$ and compute $\nabla h_i(\mathbf{w})$?

Or, pick a random subset of functions from $\{h_i(\mathbf{w})\}_{i=1}^m$ (e.g., 1%) and compute their gradients and take average? (“mini-batch gradient”)

The “gradient” computed in this way is not exact. But it works very well in practice.

Stochastic Gradient Descent (SGD)

Intuition of picking a random function $h_i(\mathbf{w})$ among $\{h_i(\mathbf{w})\}_{i=1}^m$

“Average Direction”: $\mathbb{E} \left[\nabla h_i(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) \right] \approx \frac{1}{m} \sum_{i=1}^m \nabla h_i(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) = \nabla f(\mathbf{w})$

In machine learning, we often assume that all the training data samples are independent random draws from the same joint distribution of $(\mathbf{x}, y)$

Probabilistically, every component function $h_i(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)})$ is generated in the same way. $f(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m h_i(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)})$

Randomly picking $\{\nabla h_i(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)})\}_{i \in B}$ in each iteration of the algorithm is equivalent to

say that I am going along $-\frac{1}{|B|} \sum_{i \in B} \nabla h_i(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) \approx -\nabla f(\mathbf{w}) + \text{some random noise.}$

Benefit of Stochastic Gradient Descent (SGD)

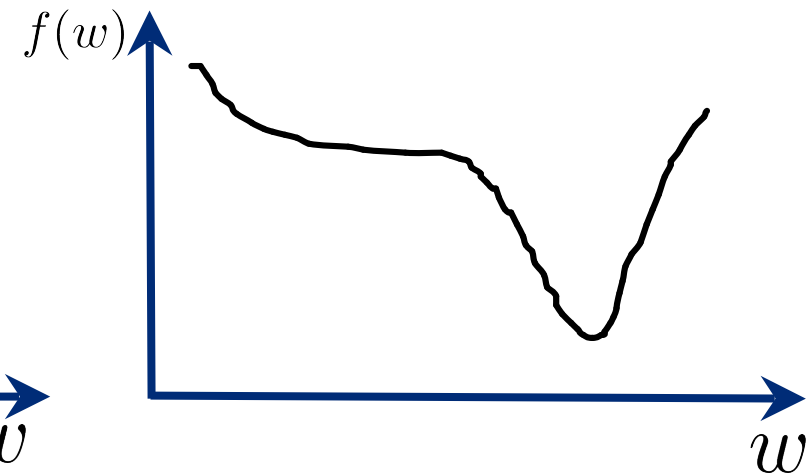
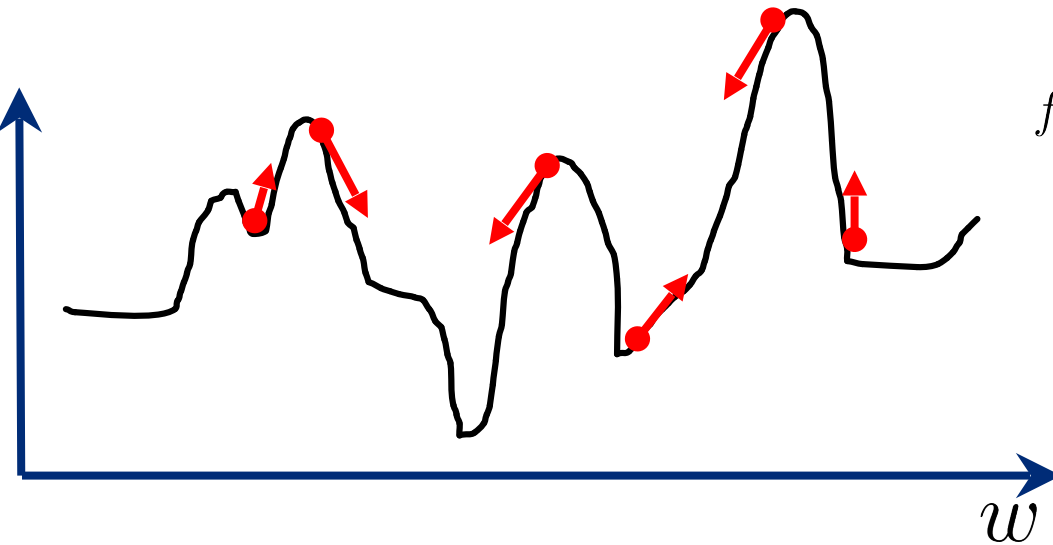
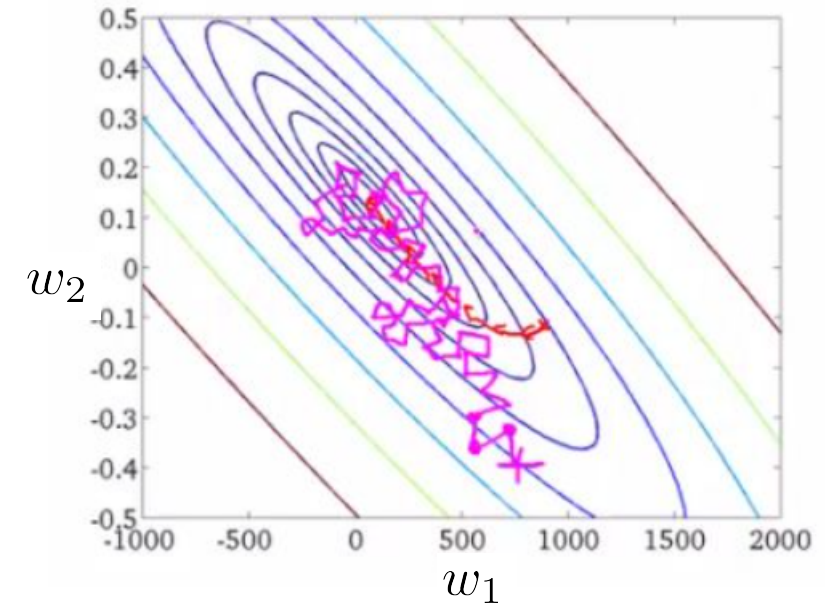
SGD is like a “drunk version” of Steepest Descent

SGD often takes more iterations to converge, but each iteration is much cheaper, so it can still be much faster overall.

1. Cheaper Computation

2. Randomization

3. Can use a pre-specified learning rate or learning-rate schedule



Other Comments about Stochastic Gradient Descent (SGD)

$$f(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m h_i(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) \quad \text{Stopping Criterion}$$

For Steepest Descent: Terminate when $\|\nabla f(\mathbf{w}(k))\| \leq \epsilon$

For SGD, pick a random subset of functions from $\{h_i(\mathbf{w})\}_{i=1}^m$ and compute their gradients and take average (“mini-batch gradient”). Hence, we no longer use $\|\nabla f(\mathbf{w}(k))\| \leq \epsilon$

We can still use $\left\| \frac{1}{R} \sum_{i=1}^R \nabla h_i(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) \right\| \leq \epsilon$, where $R < m$, provided R is very large

$$\frac{1}{R} \sum_{i=1}^R \nabla h_i(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) \approx \frac{1}{m} \sum_{i=1}^m \nabla h_i(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) = \nabla f(\mathbf{w}), \text{ if } R \text{ is large}$$

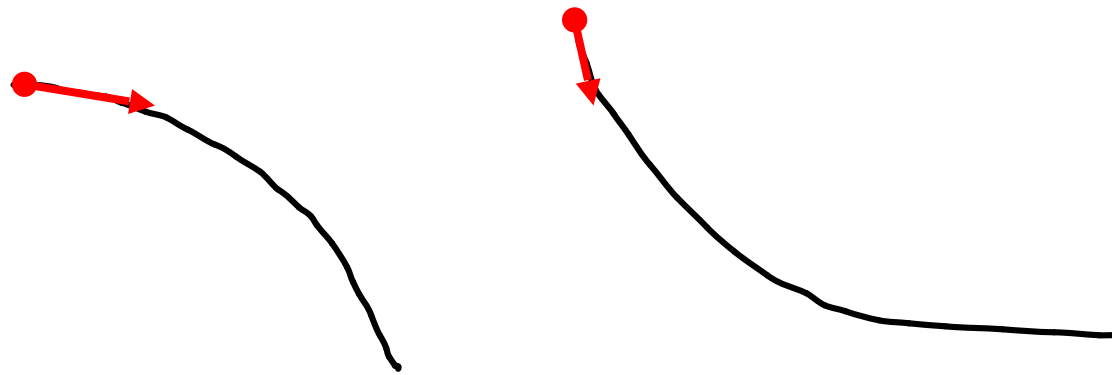
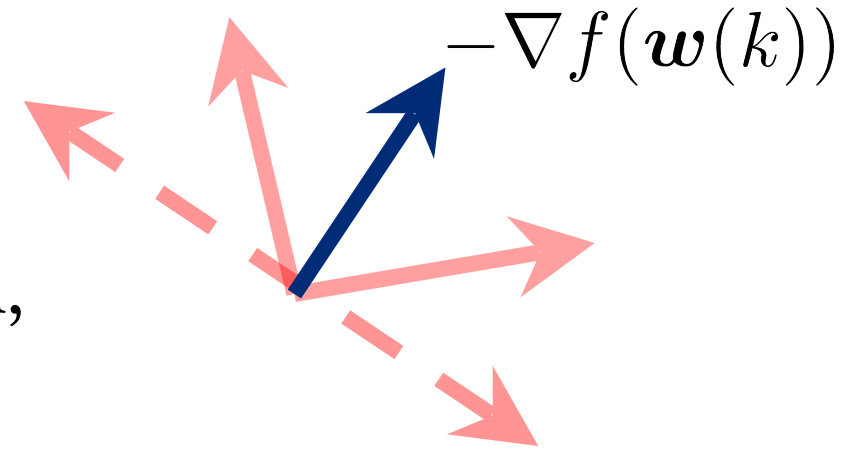
$$\|\mathbf{w}(k+1) - \mathbf{w}(k)\| \leq \epsilon$$

$$\|f(\mathbf{w}(k+1)) - f(\mathbf{w}(k))\| \leq \epsilon$$

Algorithms with Information Beyond Gradient

Gradient information (1st-order) is helpful
derive descent direction $\mathbf{p}^T(k) \cdot \nabla f(\mathbf{w}(k)) < 0$

There are other ways to identify a descent direction,
e.g., utilizing Hessian matrix (2nd-order), the
curvature of the objective function



Intuitively, the curvature (2nd-order info) can inform you how big a step you should take.

How steep the function is getting
(how fast the gradient is changing) is
reflected in the Hessian matrix

The algorithms that utilizes the 2nd-
order info can converge faster than
algorithms that only uses the 1st-order
info, sometimes

Newton's Method: Two Motivations

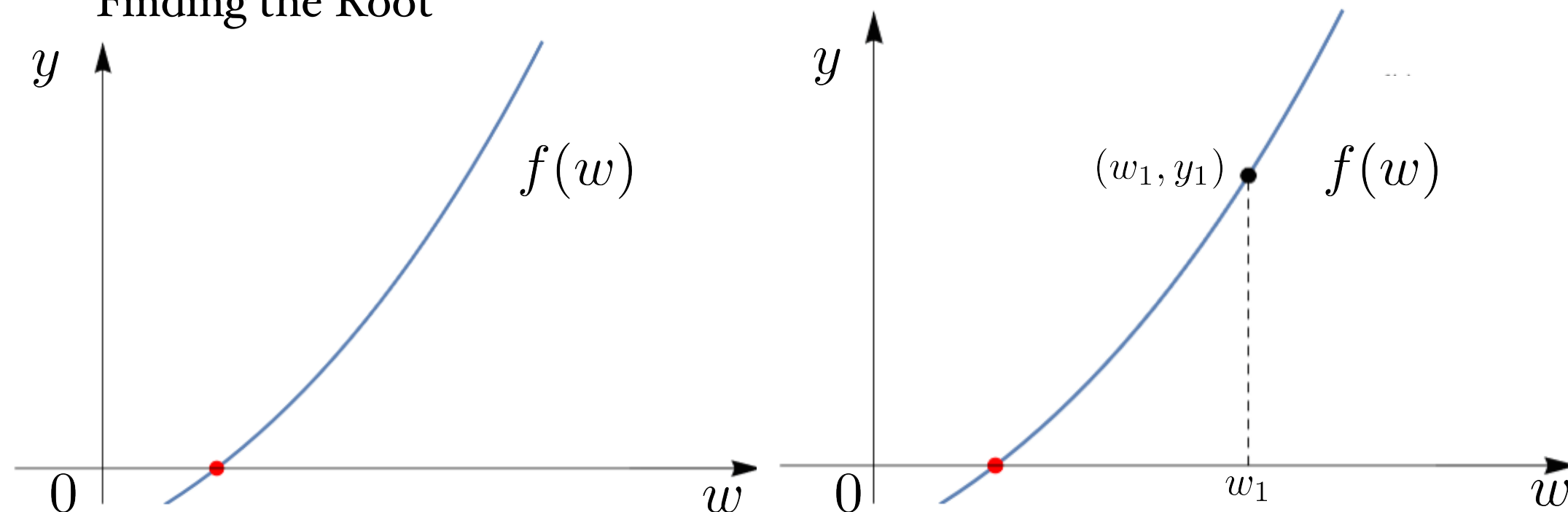
There are 2 motivations for Newton's method

1. Finding the root of equation $f(\boldsymbol{w})=0$

e.g., what is the numerical value of $\sqrt{3}$? Root of $f(w) = w^2 - 3 = 0$

2. Finding a candidate solution $\min_w f(\boldsymbol{w}) \Rightarrow \nabla f(\boldsymbol{w}) = 0$

Finding the Root



Newton's Method: Root Finding

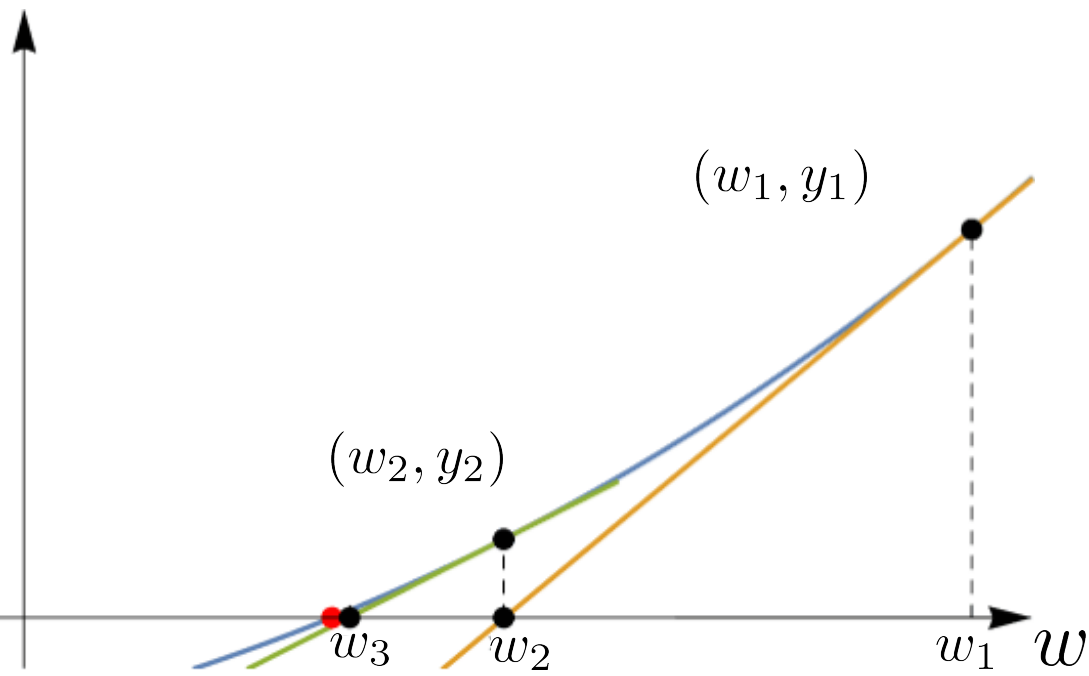
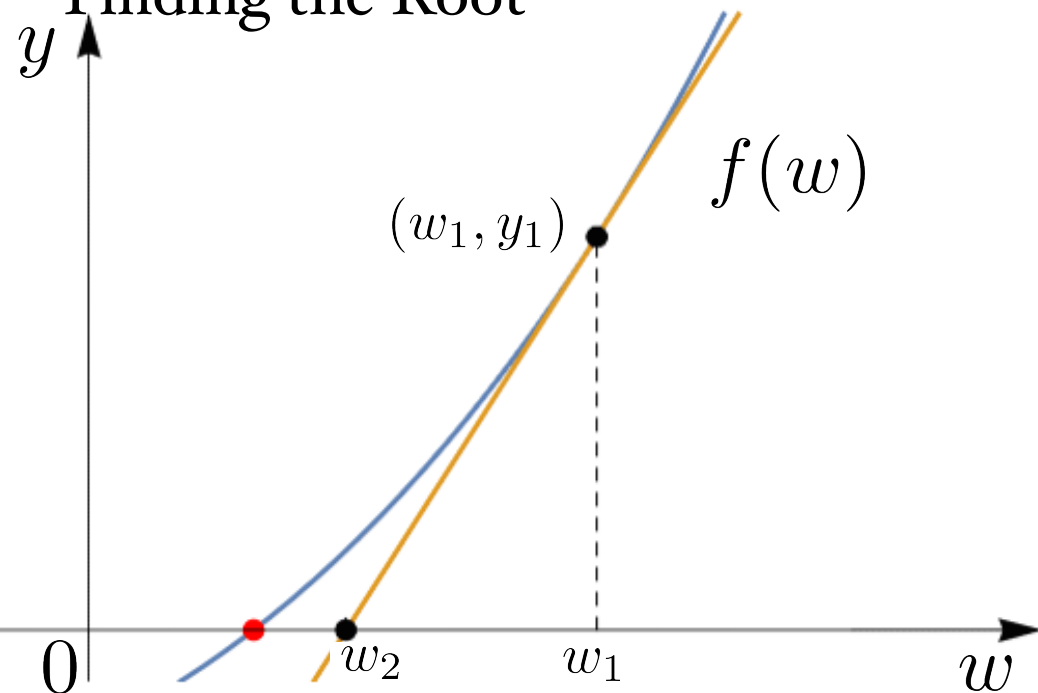
There are 2 motivations for Newton's method

1. Finding the root of equation $f(\boldsymbol{w})=0$

e.g., what is the numerical value of $\sqrt{3}$? Root of $f(w) = w^2 - 3 = 0$

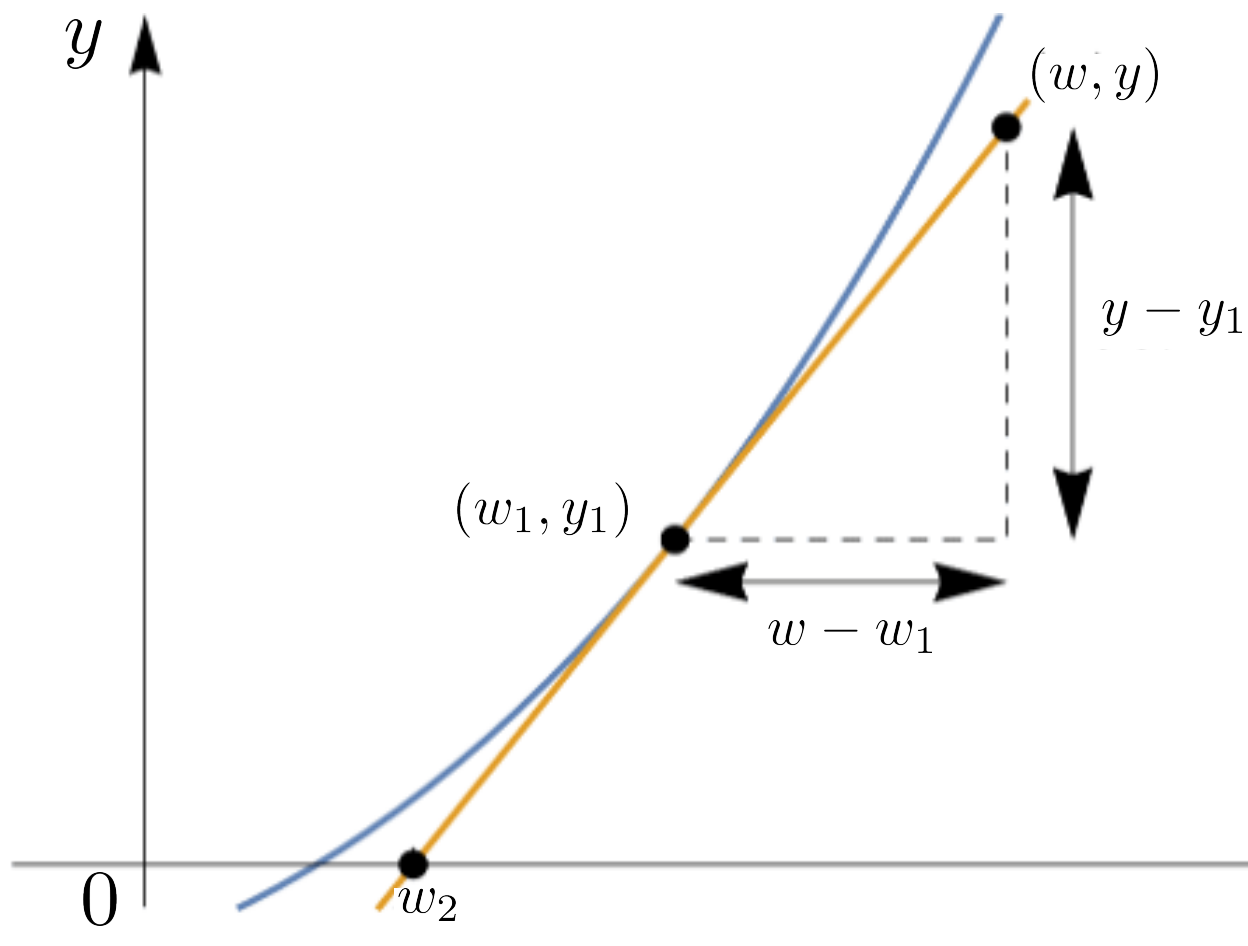
2. Finding a candidate solution to $\min_w f(\boldsymbol{w}) \Rightarrow \nabla f(\boldsymbol{w}) = 0$

Finding the Root



Newton's Method: Root Finding

The key calculation in each iteration of Newton's method is to find the root of the straight line that is tangent to $f(w)$ at the point



$$f'(w_1) = \frac{y - y_1}{w - w_1}$$



$$y = f'(w_1)(w - w_1) + y_1$$

Setting $y = 0$,

$$w_2 = w_1 - \frac{y_1}{f'(w_1)} = w_1 - \frac{f(w_1)}{f'(w_1)}$$

$$w_{k+1} = w_k - \frac{f(w_k)}{f'(w_k)} \text{ for solving } f(w) = 0$$

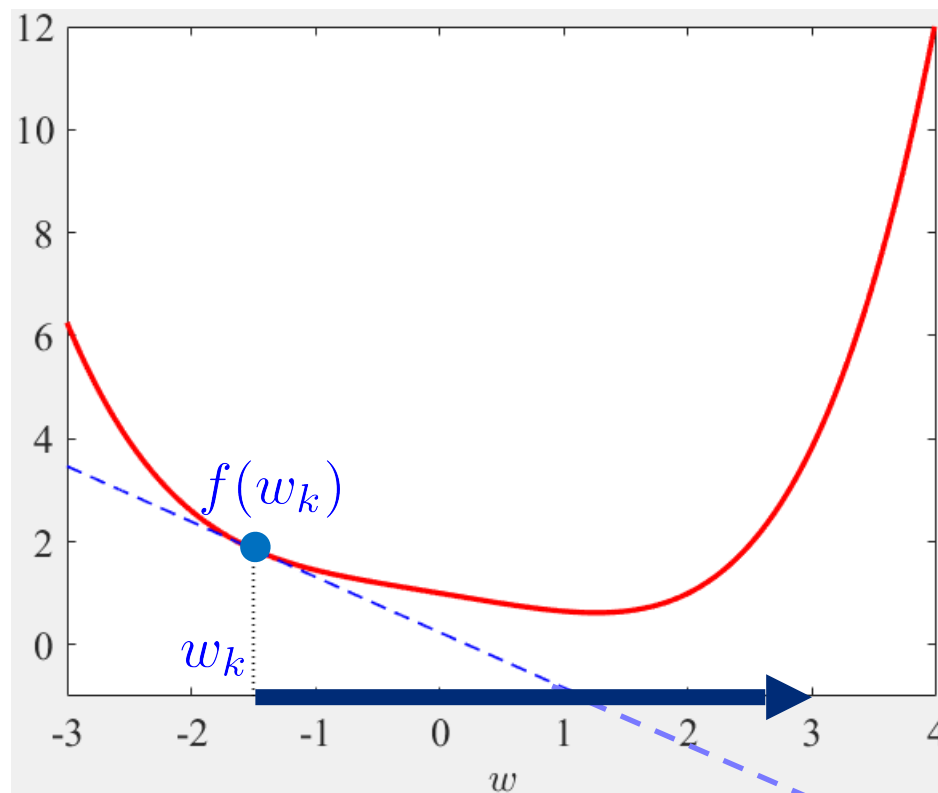
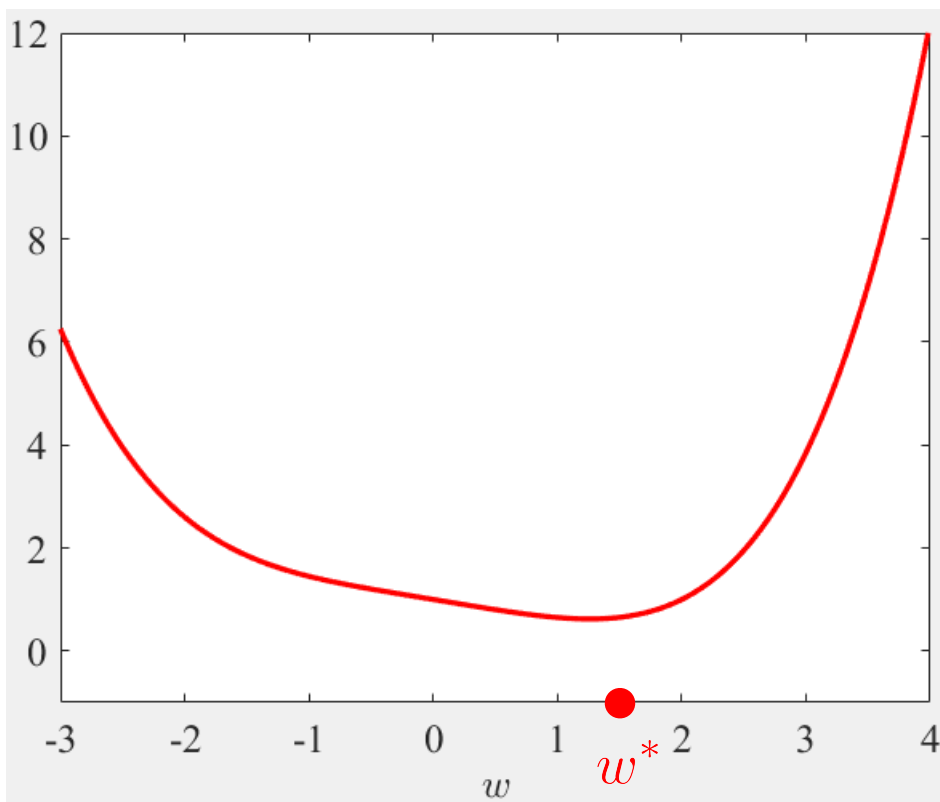
$$w_{k+1} = w_k - \frac{f'(w_k)}{f''(w_k)} \text{ for solving } f'(w) = 0$$

Newton's Method: Optimization

$w_{k+1} = w_k - \frac{f'(w_k)}{f''(w_k)}$ for solving $f'(w) = 0$, which is a necessary condition for $\arg \min_w f(w)$

Let's directly solve for $\min_w f(w)$

$$f(w) = \frac{1}{20}w^4 - \frac{2}{5}w + 1$$



$$w_{k+1} = w_k - \alpha f'(w_k)$$

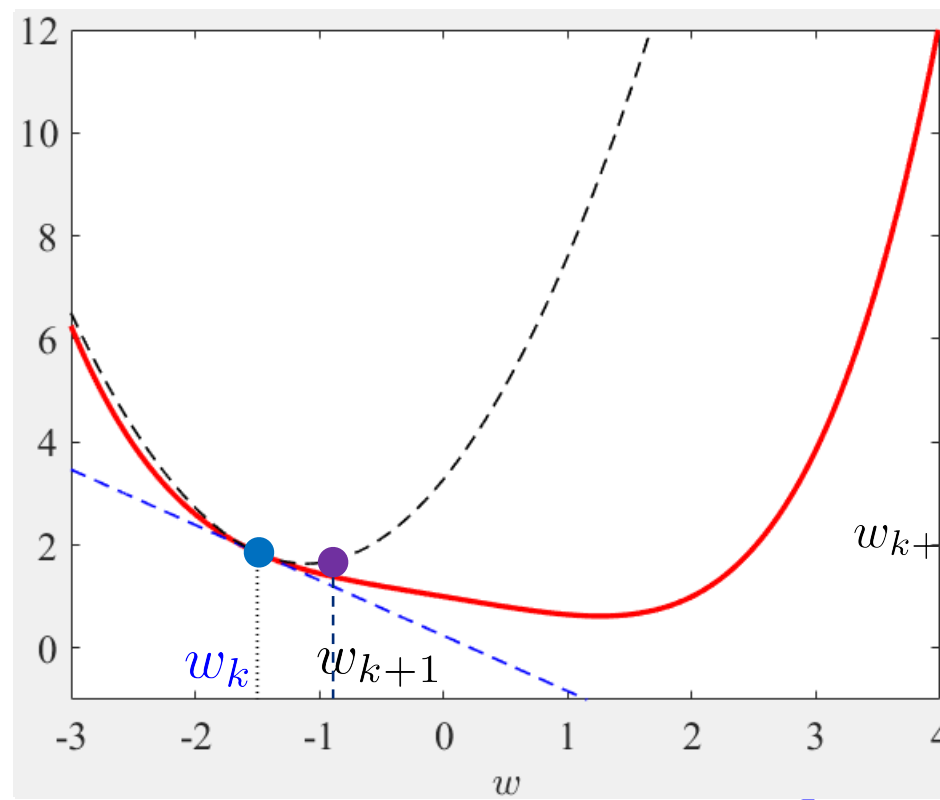
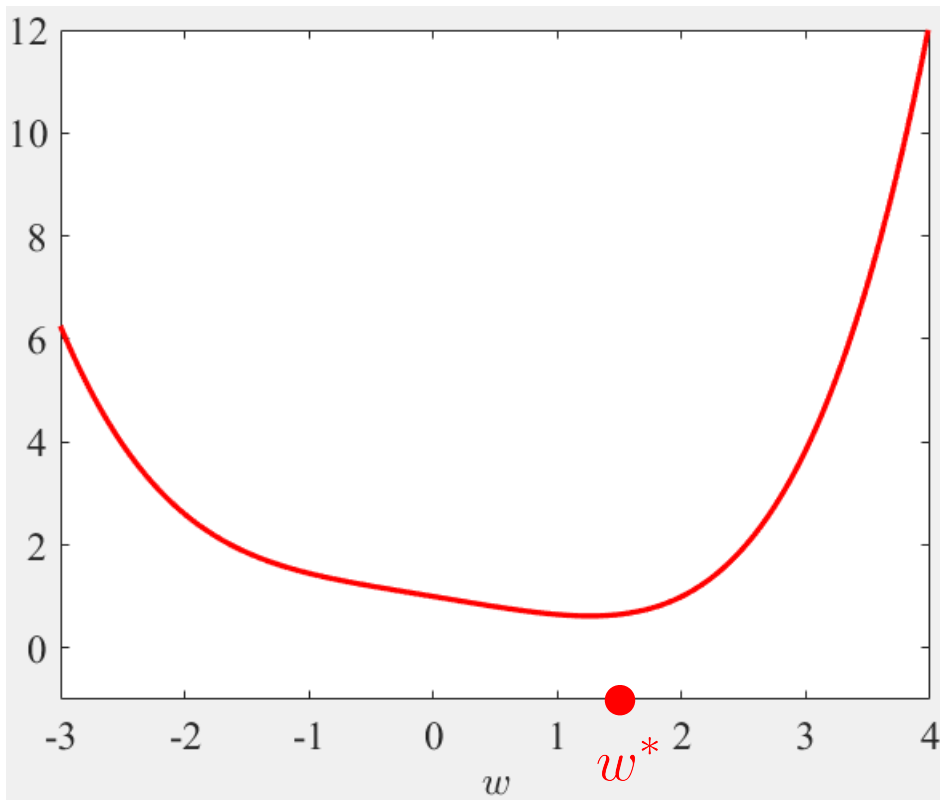
$$f(w) \approx f(w_k) + f'(w_k)(w - w_k)$$

Newton's Method: Optimization

$w_{k+1} = w_k - \frac{f'(w_k)}{f''(w_k)}$ for solving $f'(w) = 0$, which is a necessary condition for $\arg \min_w f(w)$

Let's directly solve for $\min_w f(w)$

$$f(w) = \frac{1}{20}w^4 - \frac{2}{5}w + 1$$



$$w_{k+1} = w_k - \alpha f'(w_k)$$

$$w_{k+1} = w_k - \frac{1}{f''(w_k)} f'(w_k)$$

$$f(w) \approx f(w_k) + f'(w_k)(w - w_k) + \frac{f''(w_k)}{2}(w - w_k)^2$$

Newton's Method: Optimization

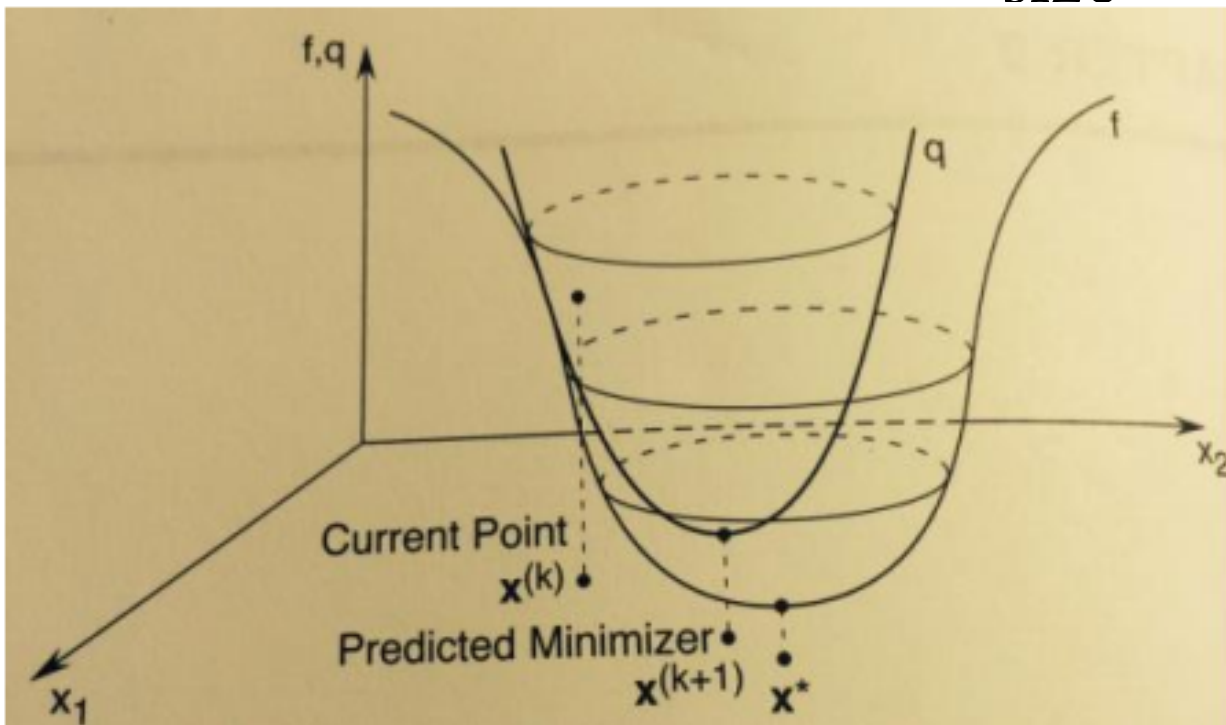
$$\mathbf{w} \in \mathbb{R}^n : f(\mathbf{w}) \approx f(\mathbf{w}_k) + \nabla f(\mathbf{w}_k)^T (\mathbf{w} - \mathbf{w}_k) + \frac{1}{2} (\mathbf{w} - \mathbf{w}_k)^T \nabla^2 f(\mathbf{w}_k) (\mathbf{w} - \mathbf{w}_k) \triangleq q(\mathbf{w})$$

For $q(\mathbf{w}) : \mathbf{w}_{k+1} = \arg \min q(\mathbf{w}) = \mathbf{w}_k - \underbrace{[\nabla^2 f(\mathbf{w}_k)^{-1}] \nabla f(\mathbf{w}_k)}_{\text{direction } \mathbf{p}_k}$, if $\nabla^2 f(\mathbf{w}_k) \succ 0$

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \frac{1}{f''(\mathbf{w}_k)} f'(\mathbf{w}_k)$$

This is the direction $\mathbf{p}_k$. Hence, the step size

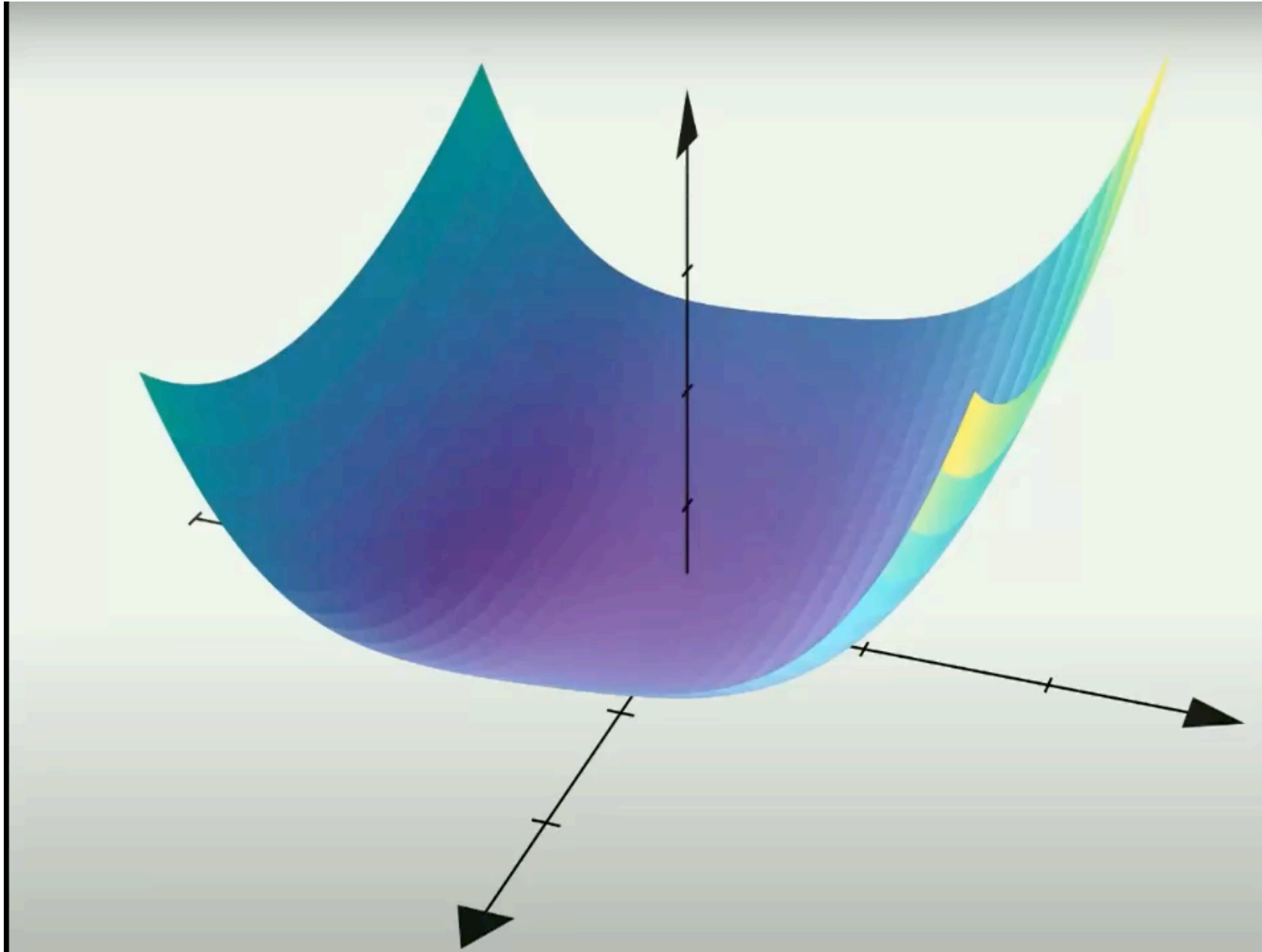
$$\alpha = 1 \begin{bmatrix} \frac{\partial^2 f}{\partial w_1^2} & \frac{\partial^2 f}{\partial w_1 \partial w_2} & \cdots & \frac{\partial^2 f}{\partial w_1 \partial w_n} \\ \frac{\partial^2 f}{\partial w_2 \partial w_1} & \frac{\partial^2 f}{\partial w_2^2} & \cdots & \frac{\partial^2 f}{\partial w_2 \partial w_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial w_n \partial w_1} & \frac{\partial^2 f}{\partial w_n \partial w_2} & \cdots & \frac{\partial^2 f}{\partial w_n^2} \end{bmatrix}$$



The greatest advantage of Newton's method is: it can achieve quadratic convergence when you get close to the minimum point

$$\nabla^2 f(\mathbf{w}_k)$$

Newton's Method: Optimization



Problems of Newton's Method

The fast convergence comes with a price

The objective function may not be twice-differentiable

The computation and storage (for Hessian related task) can be costly

	$\nabla^2 f$	$\nabla^2 f^{-1}$
n variables	n^2	$\sim n^3$
10	100	1k
10,000	100 M	1 T

Steepest Descent

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha \nabla f(\boldsymbol{w}(k))$$

Newton

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - [\nabla^2 f(\boldsymbol{w}(k))^{-1}] \nabla f(\boldsymbol{w}(k))$$

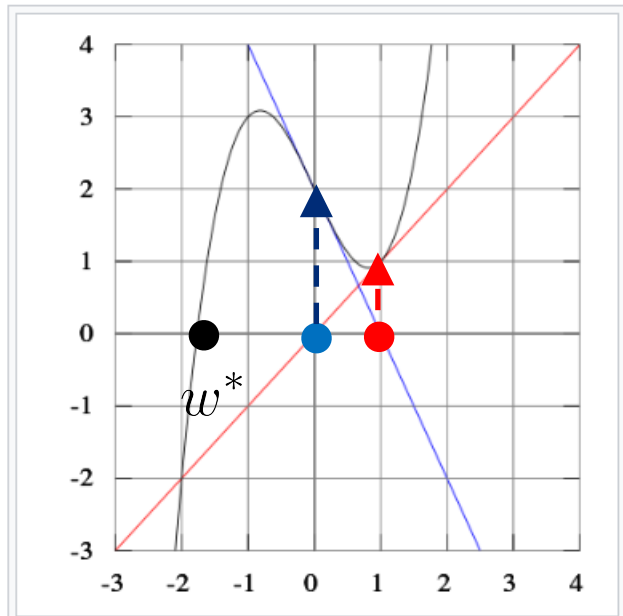
Problems of Newton's Method

What if I am far away from the stationary point?

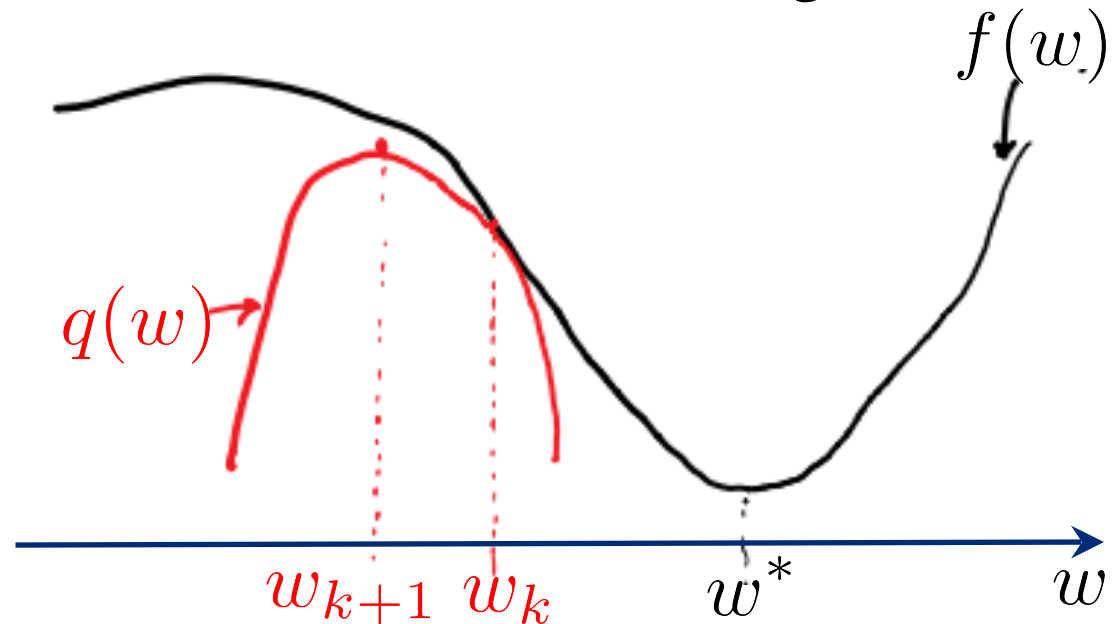
How do I know that I am close to a local minimizer? In what sense?
I don't even know where a local minimizer is!

It can generate alternating sequence that never converge

$$g(w) = w^3 - 2w + 2$$



It can move in the wrong direction



Problems of Newton's Method

$$\mathbf{w}(k+1) = \mathbf{w}(k) - \underbrace{[\nabla^2 f(\mathbf{w}(k))]^{-1} \nabla f(\mathbf{w}(k))}_{\text{Newton's direction } \mathbf{p}_k}$$

Newton's direction $\mathbf{p}_k$, **is not always a descent direction**

$$\nabla f_k^T \cdot \mathbf{p}_k < 0$$

$$-\nabla f_k^T \cdot \nabla f_k = -\|\nabla f_k\|^2 < 0$$

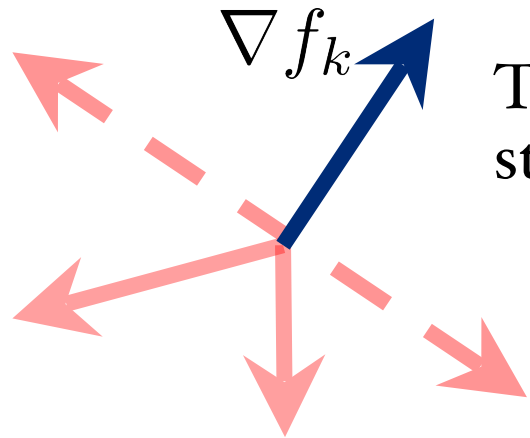
$$-\nabla f_k^T \cdot [\nabla^2 f_k^{-1}] \nabla f_k < 0?$$

if $\nabla^2 f_k^{-1}$ is positive definite

$$\mathbf{w}(k+1) = \mathbf{w}(k) - \underbrace{\alpha \nabla f(\mathbf{w}(k))}_{\text{negative gradient direction}}$$

The negative gradient is the steepest descent direction

Although the local minimum point (and points in a near neighborhood) has a positive definite Hessian, other points of the objective function may not have this property and hence, the Newton's direction may not be a descent direction



The initial point matters for Newton's method. In particular, it does not admit a global convergence property.

Newton's Method

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - [\nabla^2 f(\boldsymbol{w}(k))]^{-1} \nabla f(\boldsymbol{w}(k))$$

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - [\nabla^2 f(\boldsymbol{w}(k))]^{-1} \nabla f(\boldsymbol{w}(k))$$

Fast convergence (quadratic) when it is near a local minimum point

How do I know if I am near a local minimum point (the Newton's direction is not always descent)

Relatively costly to compute

Question: Can we capture the best of both worlds and mitigate at least to some extent the cost and other disadvantages of Newton's method?

Steepest Descent

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha \nabla f(\boldsymbol{w}(k))$$

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha \cdot \boldsymbol{I} \cdot \nabla f(\boldsymbol{w}(k))$$

Slow convergence (linear, at best)

The direction is always descent

Relatively cheap to compute

Comparison Between Newton's Method and Steepest Descent

Newton's Method

$$\mathbf{w}(k+1) = \mathbf{w}(k) - [\nabla^2 f(\mathbf{w}(k))]^{-1} \nabla f(\mathbf{w}(k))$$

$$\mathbf{w}(k+1) = \mathbf{w}(k) - [\nabla^2 f(\mathbf{w}(k))]^{-1} \nabla f(\mathbf{w}(k))$$

$$\begin{bmatrix} \frac{\partial^2 f}{\partial w_1^2} & \frac{\partial^2 f}{\partial w_1 \partial w_2} & \cdots & \frac{\partial^2 f}{\partial w_1 \partial w_n} \\ \frac{\partial^2 f}{\partial w_2 \partial w_1} & \frac{\partial^2 f}{\partial w_2^2} & \cdots & \frac{\partial^2 f}{\partial w_2 \partial w_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial w_n \partial w_1} & \frac{\partial^2 f}{\partial w_n \partial w_2} & \cdots & \frac{\partial^2 f}{\partial w_n^2} \end{bmatrix}^{-1}$$

Steepest Descent

$$\mathbf{w}(k+1) = \mathbf{w}(k) - \alpha \nabla f(\mathbf{w}(k))$$

$$\mathbf{w}(k+1) = \mathbf{w}(k) - \alpha \cdot \mathbf{I} \cdot \nabla f(\mathbf{w}(k))$$

$$\begin{bmatrix} 1 & 0 & \mathbf{0} & 0 \\ 0 & 1 & \mathbf{0} & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \mathbf{0} & 1 \end{bmatrix}$$

Quasi-Newton Methods

$$\mathbf{w}(k+1) = \mathbf{w}(k) - \alpha \cdot \mathbf{H}_k \cdot \nabla f(\mathbf{w}(k))$$

Hope for: (1) $\mathbf{H}_k$ gives me better convergence than $\mathbf{I}$

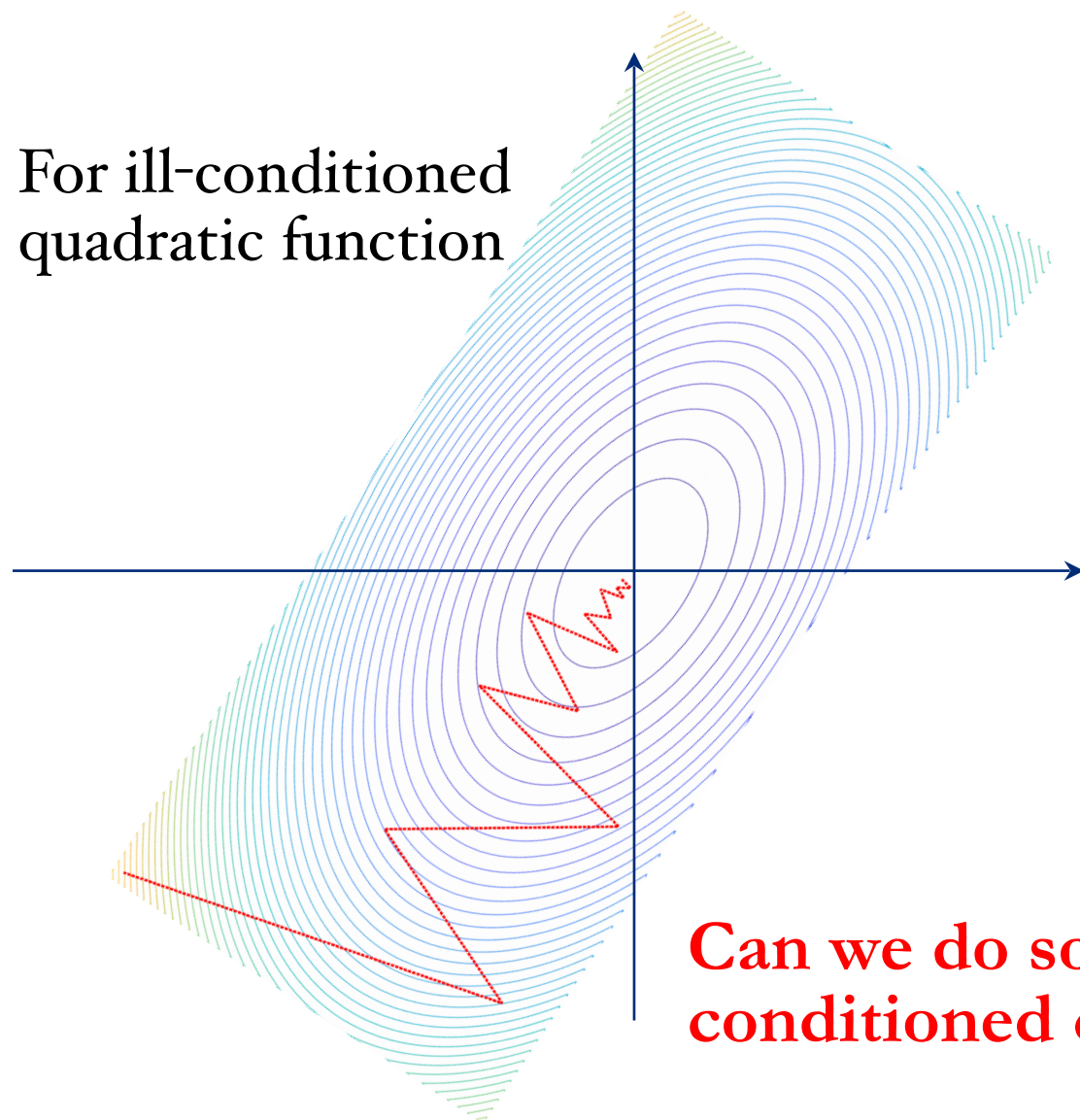
(2) $\mathbf{H}_k$ is less expensive than inverse Hessian

Comparison Between Newton's Method and Steepest Descent

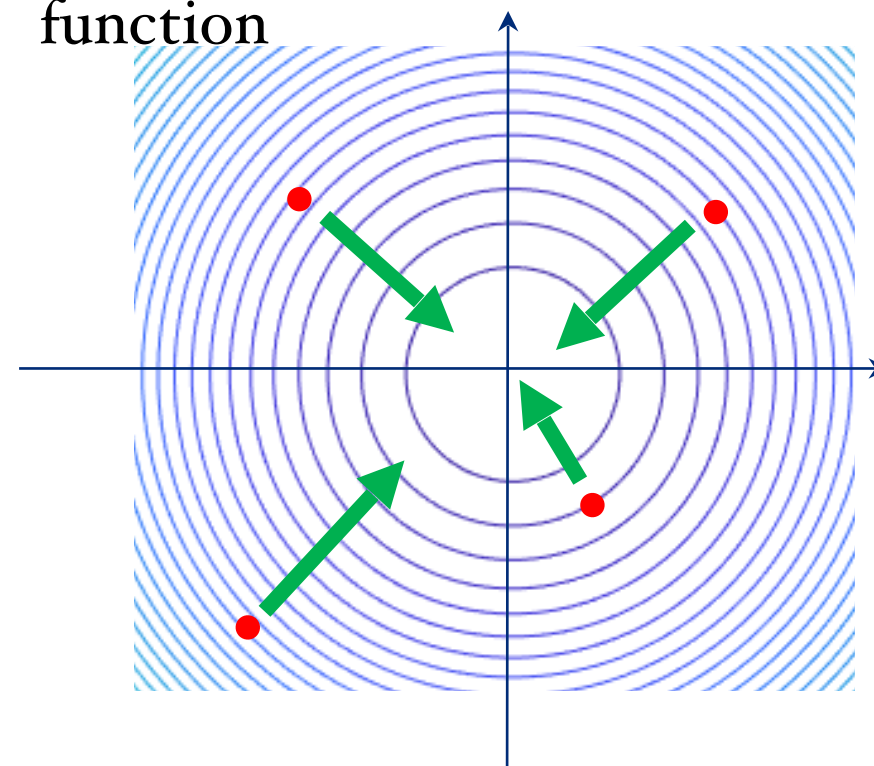
Steepest Descent

$$w(k+1) = w(k) - \alpha \cdot \boldsymbol{I} \cdot \nabla f(w(k))$$

For ill-conditioned quadratic function



For perfectly-conditioned quadratic function



Can we do some clever rescaling to transform ill-conditioned contour curves into a better one?

Steepest Descent After Transformation

$$\min_{\boldsymbol{w}} f(\boldsymbol{w}), \text{ smooth, convex}$$

Rescale space: $\boldsymbol{w} = A\boldsymbol{u}$, A is positive definite

$$\boldsymbol{u} = A^{-1}\boldsymbol{w}$$

Define: $g(\boldsymbol{u}) = f(\boldsymbol{w}) = f(A\boldsymbol{u})$

$$\boldsymbol{u}^* = \arg \min_{\boldsymbol{u}} g(\boldsymbol{u}) \qquad \boldsymbol{w}^* = \arg \min_{\boldsymbol{w}} f(\boldsymbol{w})$$

Question: $g(\boldsymbol{u}^*) = f(\boldsymbol{w}^*)$? **Yes**

$$\boldsymbol{w}^* = A\boldsymbol{u}^*$$

$$A^{-1}\boldsymbol{w}^* = \boldsymbol{u}^*$$

Question: If using Steepest Descent as minimization algorithm, is it equally hard/easy to minimize $g(\boldsymbol{u}), f(\boldsymbol{w})$? Or could one possibly be easier?